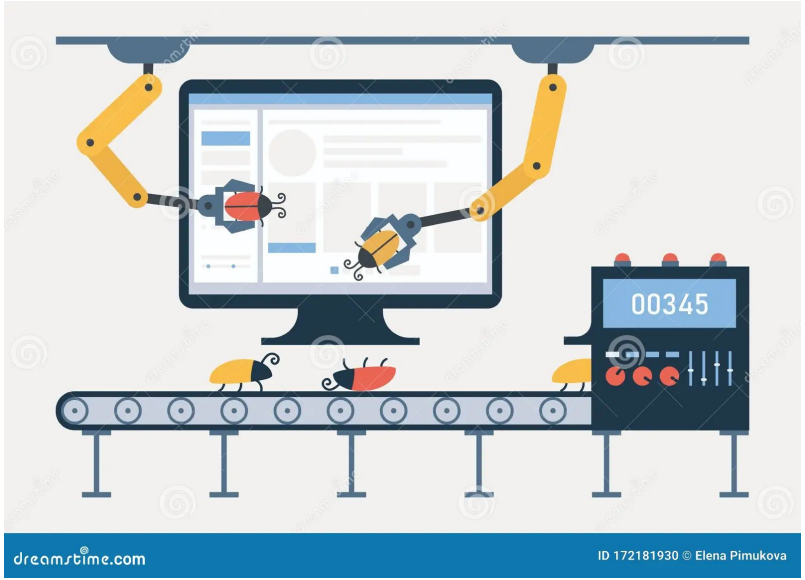
COMS W4156 Advanced Software Engineering (ASE)

September 26, 2023

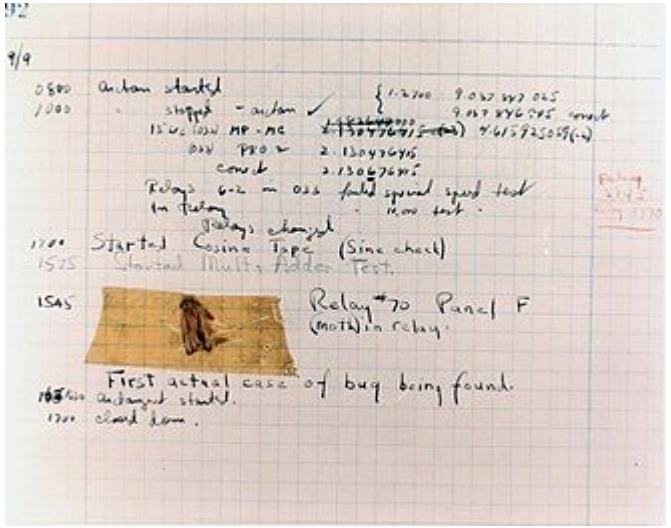
Agenda

1. Finding Bugs by executing the code
2. Choosing Test Inputs
3. Evaluating Test Outputs
4. Testing Granularities
5. Dynamic Analysis that isn't testing



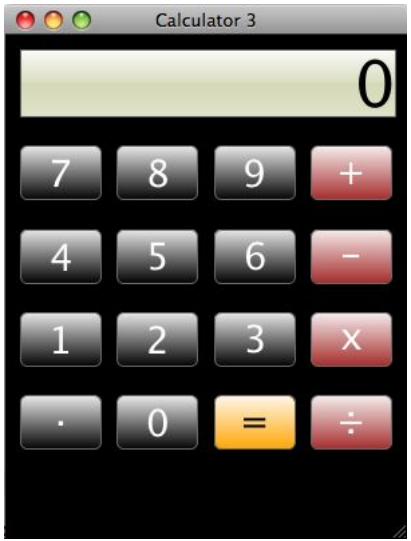
Why Do We Call Them Bugs?

First Computer Bug



Jargon File

What Kinds of Bugs Can Be Found By Executing the Code



The tester (or test script) enters a number, presses +, enters another number, presses =

- Nothing happens
- Computes the wrong answer

Calculator works correctly for some period of time or some number of computations, and then does nothing

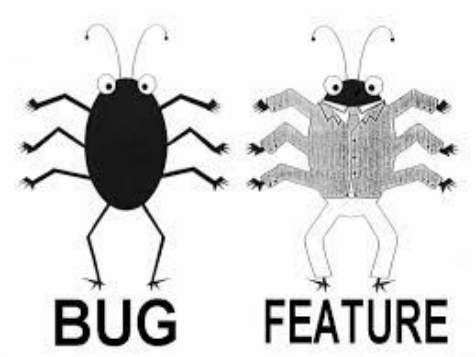
Calculator displays all 0's and won't do anything else

The calculator correctly adds, subtracts, multiplies and divides, but the tester found that if two operators are held down simultaneously, it appears to do square root

The calculator's buttons are too small, the = key is in an odd place, the display is hard to read, ...

What Kinds of Bugs Can Be Found By Executing the Code

- Software doesn’t do something requirements say it should do
- Software does something requirements say it shouldn’t do
- Software does something that requirements don’t mention
- Software doesn’t do something that requirements don’t mention but should
- Software is difficult to understand, hard to use, slow, etc.



Is This a Bug or a Feature?

The tester (or test script) enters a number, presses +, enters another number, presses =

- Nothing happens
- Computes the wrong answer

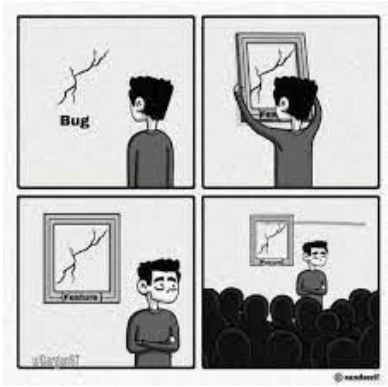
Calculator works correctly for some period of time or some number of computations, and then does nothing

Calculator displays all 0's and won't do anything else

The calculator correctly adds, subtracts, multiplies and divides, but the tester found that if two operators are held down simultaneously, it appears to do square root

The calculator's buttons are too small, the = key is in an odd place, the display is hard to read, ...

Probably a bug



Bug - memory or other resource leak; Feature - trial period over, now you have to pay for it

Bug - code stuck in infinite loop; Feature - battery low

Undocumented feature. 'Exploratory testing' = "what happens when I do this?" often finds bugs not found by planned tests

Usability bug. The user will consider these bugs even though the developers may not

Test-to-Pass

Initial testing makes sure the software minimally works with typical valid inputs ('smoke testing')

Don't push its capabilities. Don't see what you can do to break it. Treat it with kid gloves, applying the simplest and most straightforward test cases ('happy path')

Test-to-Pass 2

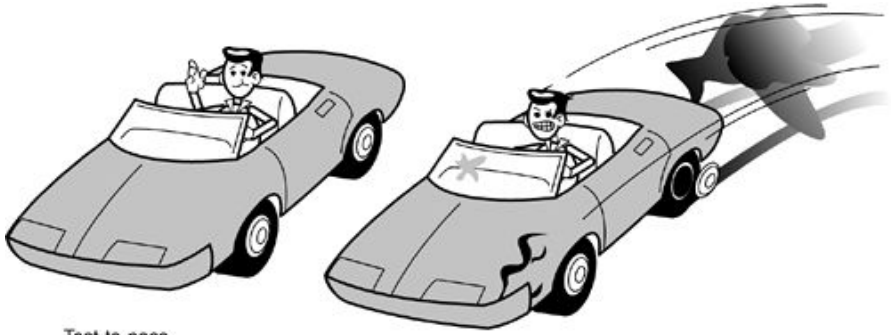
Consider testing a newly designed car

You are assigned to test the very first prototype that has just rolled off the assembly line and has never been driven

You probably wouldn't get in, start it up, head for the test track, and run it wide open at full speed as hard as you could

You could crash and die

With a new car, there could be all kinds of bugs that would reveal themselves at low speed under normal driving conditions: Maybe the tires aren't the right size, or the brakes are inadequate, or the engine is too loud. You could discover these problems and have them fixed before getting on the track and pushing the limits



Test-to-Fail

Most testing is intended to reveal bugs, so they can be fixed, which means testing with inputs designed to trigger 'corner cases'

Choose tests strategically to probe for weaknesses in the software

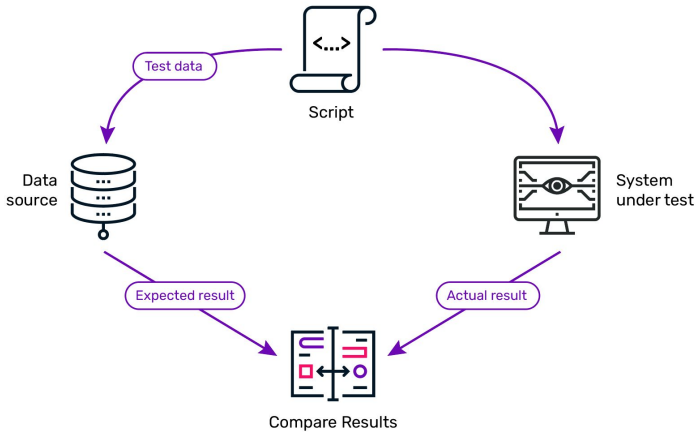
Attempt to force error messages, e.g., saving a file to a flash drive when there isn't one inserted

The documentation may state that certain input conditions should result in an error message - testing with these input conditions is arguably either test-to-pass or test-to-fail

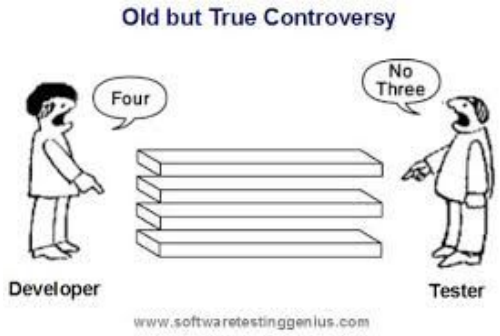
But also invent test cases to try to force errors that are not documented

How to Choose Test Inputs

Typical valid inputs	Enter age: _____25_____
Atypical valid inputs	Enter age: _____116_____
Invalid inputs	Enter age: _____3.14159_____



Invalid Input Formats



String inputs with valid vs. invalid *formatting*

- UTF-8 when ASCII is required (and vice versa)
- Printing vs. non-printing characters (e.g., control characters)
- Upper vs. lower case
- Space, tab, newline characters (whitespace)
- Any characters with special meaning to the application (or to the implementation, e.g., programming language, SQL)
- Null/empty string
- Any string above maximum buffer size = "buffer overflow"
  (the buffer is the data storage for reading the string input)

Where do Invalid Inputs come from?

Users

Network

Devices

Databases

Files

...

Example: Choosing Test Inputs

Consider a simple calendar program where the user can enter an arbitrary date and the program returns the corresponding day of the week. The user is prompted to enter the month, the day, and the year in separate fields

The calendar represents months as integers 1 to 12, days as integers 1 to 31, and years as integers 1 to 9999 (AD)

How should we choose test cases for this program?

Example: Choosing Test Inputs 2

Is there anything special about the days 29, 30 or 31 that we should test?

Should we test the month, day, or year 0?

What about testing with negative integers?

Should we test with month 13? What about day 32?

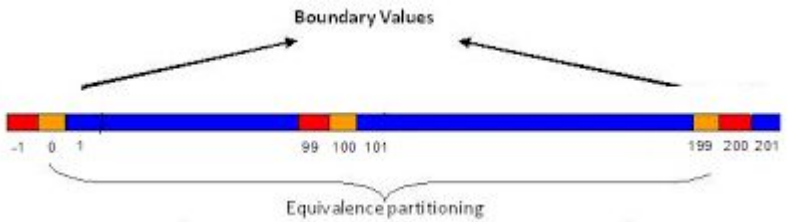
Should we test with non-numeric input?

➢ How do we know whether the test outputs are correct?

Choosing Test Inputs for your Team Projects

There will be a lot more material about how best to choose test inputs coming soon, including 'equivalence partitions' and 'boundary analysis'

You will be expected to consider equivalence partitions and boundary analysis, as well as at least one each typical valid, atypical valid and invalid inputs for both unit and system testing (testing granularities also coming soon)



Test Oracles

The entity that knows the correct outputs, and/or can check that the actual outputs are the same as or consistent with the expected outputs, is called the 'test oracle'

Imagine the human tester is the test oracle and knows what the correct output should be for each input, so they write tests with assertions to check for these expected results. That works for some programs and some inputs...

But… do you know the day of the week for January 1, 0001? How about September 14, 3333?

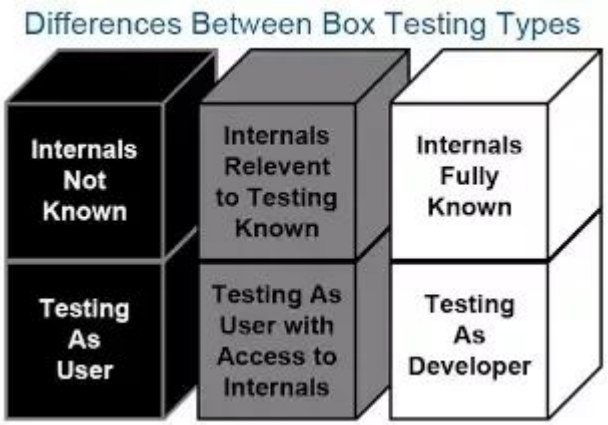
What would you do if you did not have Google or ChatGPT to ask? More on test oracles later in semester

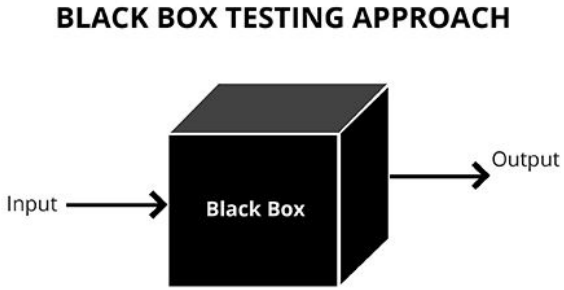
Test Oracles for your Team Projects

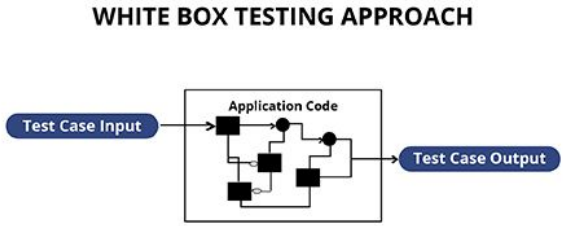
We do not expect you to use any special test oracle tools, but you should include automated assertions in your test cases when feasible

Approaches To Testing

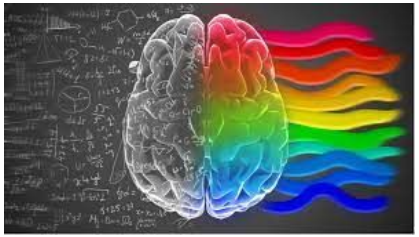
Black box - tester considers what the box is supposed to do, not how it does it, doesn't look inside the box







Testing is not very Colorful



Gray box (or grey box) - tester looks at intermediate products between boxes (e.g., network I/O) and leftover side-effects after box is done executing (e.g., temporary files, database connections left open)

Some bugs are not immediately visible externally. Check logs, databases, file system, network traffic, devices, …

White box - tester leverages full knowledge of what is inside the box

➢ If you never ran this part of the code, how can you have any confidence that it works?

Helps with choosing inputs intended to reach specific parts of the code

Unit testing is often implicitly white box, since typically done by the same developers who coded the box

Coverage

At minimum, unit testing and system testing collectively exercise every statement - much easier to force with unit than system testing

Better: Exercise every branch

Beware the missing else!

100% coverage may check missing conditional cases but can never detect entirely missing code

"Call off the search. I don't miss him anymore."

```
if (condition) { do something }
else { do something else }
```

```
if (condition) { do something }
```

Alternative Version of Coverage Slide

At minimum, unit testing, integration testing and system testing collectively exercise every statement

Better: Exercise every branch (or >= 80% of branches)

Coverage may check missing conditional cases but can never detect entirely missing code

Beware the missing else!

```
if (condition) { do something }
else { do something else }
// some other code is here
// some other code should be here but isn't

if (condition) { do something }
// some other code is here
// some other code should be here but isn't
```

Greybox and Whitebox Testing for your Team Projects

You are not required to do full greybox testing, but you need to check your project's datastore as well as any other potentially modified databases, files, or other side-effects on your service's environment

You are expected to use a branch coverage tool targeting your chosen programming language and try to exercise at least 80% of your codebase

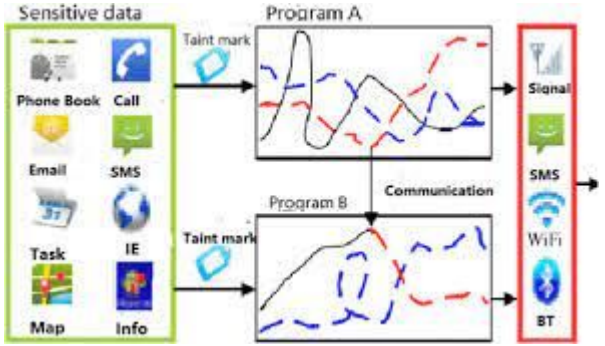
What is the Difference between Dynamic Analysis and Testing?

Any program analysis technique that executes the code is 'dynamic'

Some dynamic analyzers don't need to know what your code is supposed to do, similar to static analyzers, but can still find bugs:

Model checking has several different meanings, for the purposes of this course I mean testing that the code handles every possible return value from every API it uses, aka API testing but for the APIs you use not just your own

Taint tracking marks sensitive data at its source,
traces its path through program, and detects unexpected sinks



Fuzzing takes existing valid test cases and 'fuzzes' the inputs with semi-random changes, seeking to produce inputs that are

- structurally invalid
- structurally valid but semantically invalid

Dynamic Analysis for your Team Projects

When you implement your sample client in the second iteration, you should do model checking for your own API from that client

You are not required to do model checking for external APIs, taint tracking, or fuzz testing, but we expect you to write and run a LOT of tests with invalid as well as valid inputs for every external API entry point used by your service PLUS every API entry point provided by your service. If you would like to try a dynamic analysis tool, go for fuzzing, e.g., Google's OSS-Fuzz - it WILL find bugs!

Upcoming Assignments

- ➔ Team Revised Project Proposal due next Monday, October 2, by 11:59pm
- ➔ First Iteration due Monday, October 16, by 11:59pm
- ➔ First Iteration Demo due Monday, October 23, by 11:59pm

Thursday

More on APIs: Documentation and Testing

Ask Me Anything